ADVANCED MACHINE LEARNING

Comprehensive Study Material

Units 1 & 2 | Complete Notes with Q&A

Topics Covered:

Unit 1: Advanced Machine Learning Foundations

Unit 2: Advanced Supervised Learning Techniques

50 Q&A | 10-Mark, 5-Mark & 2-Mark Questions

Table of Contents

UNIT 1: ADVANCED MACHINE LEARNING FOUNDATIONS

1.1 Review of Fundamental ML Concepts

1.2 Bias-Variance Trade-off and Overfitting

1.3 Regularization Techniques

1.4 Cross-Validation Strategies

1.5 Ensemble Learning Methods

UNIT 2: ADVANCED SUPERVISED LEARNING TECHNIQUES

2.1 Support Vector Machines (SVMs) and Kernel Methods

2.2 Decision Tree Ensembles

2.3 Deep Learning Architectures

2.4 Transfer Learning and Domain Adaptation

2.5 Multi-task Learning and Meta-learning

QUESTION & ANSWER SECTION

Section A: 10-Mark Questions (Q1–Q15)

Section B: 5-Mark Questions (Q16–Q35)

Section C: 2-Mark Questions (Q36–Q50)

UNIT 1: Advanced Machine Learning Foundations

1.1 Review of Fundamental Machine Learning Concepts

Machine learning (ML) is a branch of artificial intelligence that enables computers to learn from data without being explicitly programmed for each task. ML algorithms identify patterns, make decisions, and improve predictions through experience. The three primary paradigms of machine learning are supervised learning, unsupervised learning, and reinforcement learning.

1.1.1 Supervised Learning

In supervised learning, the algorithm is trained on a labeled dataset where each training example consists of an input feature vector and a corresponding target label. The model learns a mapping function $f: X \rightarrow Y$ from inputs X to outputs Y.

Key characteristics of supervised learning:

- Training data consists of input-output pairs (x_i, y_i)
- Goal: Minimize prediction error on unseen data
- Applications: Image classification, spam detection, medical diagnosis, stock prediction
- Types: Classification (discrete outputs) and Regression (continuous outputs)

Key Formula: Empirical Risk Minimization

The goal is to find parameters θ that minimize the loss function: $L(\theta) = (1/n) \Sigma\, l(f(x_i; \theta), y_i)$, where l is the per-sample loss (e.g., cross-entropy for classification, MSE for regression).

1.1.2 Unsupervised Learning

Unsupervised learning deals with unlabeled data. The goal is to discover hidden structure, patterns, or representations in the data without explicit guidance.

- Clustering: Grouping similar data points (K-means, DBSCAN, Hierarchical)
- Dimensionality Reduction: PCA, t-SNE, UMAP — compressing high-dim data
- Density Estimation: Learning the probability distribution of data
- Generative Models: VAEs and GANs that can generate new samples

1.1.3 Reinforcement Learning

Reinforcement learning (RL) involves an agent learning to make sequential decisions by interacting with an environment. The agent receives rewards or penalties based on its actions and learns a policy to maximize cumulative reward.

Core components of RL:

- Agent: The learner/decision-maker
- Environment: What the agent interacts with
- State (S): Current situation of the agent
- Action (A): Set of all possible moves
- Reward (R): Feedback from the environment
- Policy (π): Strategy mapping states to actions

- Value Function V(s): Expected cumulative reward from state s

1.2 Bias-Variance Trade-off and Overfitting

The bias-variance trade-off is one of the most fundamental concepts in machine learning theory. It describes the relationship between a model's complexity, its training performance, and its generalization ability.

1.2.1 Decomposition of Prediction Error

Bias-Variance Decomposition

Total Error = Bias² + Variance + Irreducible NoiseBias: Error from wrong assumptions in the model (underfitting)Variance: Error from sensitivity to small fluctuations in training data (overfitting)Irreducible Noise: Inherent noise in the data that cannot be reduced

1.2.2 High Bias (Underfitting)

A model with high bias is too simple to capture the underlying patterns in the data. It performs poorly on both training and test data. Common causes include: using a linear model for non-linear data, insufficient model capacity, and excessive regularization.

1.2.3 High Variance (Overfitting)

A model with high variance is overly complex and learns the noise in the training data rather than the underlying pattern. It performs well on training data but poorly on new data. Common causes include: too many parameters, insufficient training data, and lack of regularization.

1.2.4 The Optimal Balance

The ideal model strikes a balance between bias and variance, achieving low error on both training and test sets. This is typically found through careful model selection, cross-validation, and regularization. As model complexity increases: bias decreases, variance increases. The optimal point minimizes total test error.

1.3 Regularization Techniques

Regularization refers to techniques used to prevent overfitting by adding a penalty to the model's complexity during training. These techniques constrain the model and encourage simpler solutions that generalize better.

1.3.1 L1 Regularization (Lasso)

L1 regularization adds the sum of the absolute values of the model's weights to the loss function. It promotes sparsity by driving many weights to exactly zero, effectively performing feature selection.

L1 Regularization Formula

L_total = L_data + λ Σ|wj|λ: Regularization strength (hyperparameter)Effect: Sparse solutions — many weights become exactly 0Use case: Feature selection, when only a subset of features are relevant

1.3.2 L2 Regularization (Ridge)

L2 regularization adds the sum of the squared weights to the loss function. Unlike L1, it distributes weight reduction across all parameters, rarely forcing them to exactly zero.

L2 Regularization Formula

L_total = L_data + λ Σwj²Effect: Small but nonzero weights — weight shrinkageAdvantage: Convex, easy to optimize; handles correlated features wellUse case: When all features contribute somewhat to the prediction

1.3.3 Elastic Net Regularization

Elastic Net combines both L1 and L2 regularization. It balances the benefits of both: sparsity from L1 and stability from L2. Formula: λ1Σ|wj| + λ2Σwj². Useful when there are multiple correlated features.

1.3.4 Dropout

Dropout is a regularization technique specifically designed for neural networks. During training, neurons are randomly set to zero with probability p (the dropout rate). This prevents co-adaptation of neurons and forces the network to learn more robust features.

- Training: Each neuron deactivated independently with probability p
- Inference: All neurons active, weights scaled by (1-p) or inverted dropout
- Effect: Equivalent to training an ensemble of 2^n different networks
- Typical values: p = 0.2–0.5 for hidden layers

1.3.5 Batch Normalization

Batch Normalization (BatchNorm) normalizes the input of each layer to have zero mean and unit variance within a mini-batch. This reduces internal covariate shift, accelerates training, and acts as a mild regularizer.

Batch Normalization Process

1. Compute batch mean: μ_B = (1/m) Σxi2. Compute batch variance: σ²_B = (1/m) Σ(xi - μ_B)²3. Normalize: x̂i = (xi - μ_B) / √(σ²_B + ε)4. Scale and shift: yi = γx̂i + β (learnable parameters γ, β)Benefits: Faster training, higher learning rates, reduced sensitivity to weight initialization

1.4 Cross-Validation Strategies

Cross-validation is a statistical technique for assessing the generalization performance of a machine learning model. It provides a robust estimate of model performance by evaluating the model on multiple data splits.

1.4.1 Hold-Out Validation

The dataset is split into training (typically 70–80%) and test sets (20–30%). Simple and fast but may produce high variance estimates, especially with small datasets.

1.4.2 K-Fold Cross-Validation

The dataset is divided into K equal subsets (folds). The model is trained K times, each time using K-1 folds for training and 1 fold for validation. The performance is averaged across all K runs.

- Reduces variance compared to hold-out validation
- Common values: K = 5 or K = 10
- Each example is used for validation exactly once
- Computationally expensive: requires K model trainings

1.4.3 Stratified K-Fold

A variant of K-fold where each fold maintains the same proportion of samples from each class as the complete dataset. Essential for imbalanced classification problems to ensure each fold is representative.

1.4.4 Leave-One-Out Cross-Validation (LOOCV)

A special case of K-fold where K equals the number of samples. Each sample is used as the validation set exactly once. Produces an almost unbiased estimate but is computationally very expensive for large datasets.

1.4.5 Hyperparameter Tuning with Cross-Validation

Cross-validation is essential for hyperparameter tuning. Common strategies include Grid Search (exhaustive search over parameter grid), Random Search (random sampling from parameter distributions — often more efficient), and Bayesian Optimization (uses probabilistic model to guide search toward promising configurations).

1.5 Ensemble Learning Methods

Ensemble learning combines multiple individual models (base learners) to produce a single, stronger model. The key insight is that diverse models, when combined, can often outperform any individual model. The main strategies are bagging, boosting, and stacking.

1.5.1 Bagging (Bootstrap Aggregating)

Bagging trains multiple base models independently on different bootstrap samples of the training data (random sampling with replacement). Predictions are combined by majority voting (classification) or averaging (regression).

- Reduces variance without significantly increasing bias
- Models can be trained in parallel — computationally efficient
- Random Forest is the most popular bagging algorithm
- OOB (Out-of-Bag) error provides free validation estimate

1.5.2 Boosting

Boosting trains base models sequentially, where each subsequent model focuses on correcting the errors made by previous models. Weak learners are combined into a strong learner through weighted voting.

Boosting Key Idea

Train model M1, compute residuals/errors.Train M2 to correct M1's errors. Train M3 to correct M1+M2's errors...Final prediction: $F(x) = \Sigma\ \alpha t * ht(x)$, weighted sum of base learners.Popular algorithms: AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost

1.5.3 Stacking (Stacked Generalization)

Stacking uses predictions from multiple different base models (level-0 learners) as input features for a meta-learner (level-1 learner). The meta-learner learns how to best combine the base model predictions.

- Level-0 models: Diverse algorithms (e.g., SVM, Random Forest, Neural Network)
- Level-1 (meta-learner): Often a simple model like logistic regression or linear regression
- Uses cross-validation to generate out-of-fold predictions for training meta-learner
- Can capture complex relationships between base model predictions

UNIT 2: Advanced Supervised Learning Techniques

2.1 Support Vector Machines (SVMs) and Kernel Methods

Support Vector Machines are powerful supervised learning models used for classification and regression. SVMs find the optimal hyperplane that maximally separates different classes in the feature space.

2.1.1 Linear SVM — Hard Margin

For linearly separable data, SVM finds the hyperplane $w^Tx + b = 0$ that maximizes the margin between classes. The margin is $2/||w||$, so maximizing the margin is equivalent to minimizing $||w||^2$.

SVM Optimization Problem

Minimize: $(1/2)||w||^2$Subject to: $y_i(w^Tx_i + b) \geq 1$ for all iSupport Vectors: Points closest to the hyperplane — they define the decision boundaryMargin = $2/||w||$ — maximized by SVM

2.1.2 Soft Margin SVM

For non-linearly separable data, the soft margin SVM introduces slack variables ξ_i that allow misclassification. The hyperparameter C controls the trade-off between maximizing the margin and minimizing misclassification.

Large C: Smaller margin, fewer misclassifications (low bias, high variance). Small C: Larger margin, more misclassifications tolerated (high bias, low variance).

2.1.3 Kernel Methods

Kernel methods allow SVMs to operate in high-dimensional feature spaces without explicitly computing the transformation. The kernel function $K(x_i, x_j) = \varphi(x_i)^T\varphi(x_j)$ computes inner products in the feature space implicitly.

- Linear Kernel: $K(x_i, x_j) = x_i^Tx_j$ — suitable for linearly separable data
- Polynomial Kernel: $K(x_i, x_j) = (x_i^Tx_j + c)^d$ — captures polynomial interactions
- RBF/Gaussian Kernel: $K(x_i, x_j) = \exp(-\gamma||x_i - x_j||^2)$ — most widely used
- Sigmoid Kernel: $K(x_i, x_j) = \tanh(\alpha x_i^Tx_j + c)$ — similar to neural networks

2.1.4 SVM for Regression (SVR)

Support Vector Regression fits a tube of width ε around the function. Points inside the tube incur no penalty; points outside the tube incur a loss proportional to their distance from the tube. This makes SVR robust to outliers.

2.2 Decision Tree Ensembles

2.2.1 Decision Trees — Foundation

Decision trees partition the feature space into rectangular regions using recursive binary splits. Each split selects the feature and threshold that best separates the classes (or minimizes regression error). Splits are chosen to maximize information gain (classification) or minimize MSE (regression).

- Gini Impurity: $Gini(p) = 1 - \Sigma p_k^2$ — used in CART algorithm

- Entropy: $H(p) = -\Sigma p_k \log(p_k)$ — used in ID3, C4.5 algorithms
- Variance Reduction: Used for regression trees

2.2.2 Random Forests

Random Forests extend bagging to decision trees with two key sources of randomness: (1) bootstrap sampling of training data, and (2) random feature subsets considered at each split (typically $\sqrt{p}$ features for classification, p/3 for regression).

- Hyperparameters: n_estimators, max_features, max_depth, min_samples_split
- Feature Importance: Average decrease in impurity across all trees for each feature
- OOB Error: ~37% of samples unused in each tree — used for validation
- Strengths: Handles high-dimensional data, robust to outliers, interpretable feature importance

2.2.3 Gradient Boosting Machines (GBM)

Gradient Boosting builds an ensemble of trees sequentially, where each tree is fit to the negative gradient (pseudo-residuals) of the loss function with respect to the previous model's predictions.

Gradient Boosting Algorithm

1. Initialize $F_0(x)$ = constant (e.g., mean of y)2. For t = 1 to T: a. Compute pseudo-residuals: $r_{it} = -\partial L(y_i, F(x_i))/\partial F(x_i)$ b. Fit a regression tree h_t to pseudo-residuals c. Update: $F_t(x) = F_{t-1}(x) + \eta * h_t(x)$ (η = learning rate)3. Final model: $F(x) = \Sigma \eta * h_t(x)$

2.2.4 XGBoost (Extreme Gradient Boosting)

XGBoost is an optimized, regularized implementation of gradient boosting that has become the dominant algorithm for structured data in machine learning competitions.

Key innovations of XGBoost:

- Regularization: L1 and L2 regularization on leaf weights to prevent overfitting
- Second-order optimization: Uses both gradient (first derivative) and Hessian (second derivative)
- Sparsity-aware split finding: Handles missing values and sparse data automatically
- Tree pruning: Uses max_depth and grows trees depth-first, then prunes back
- Column subsampling: Random feature selection per tree or per level
- Parallel tree construction: Uses block structure for distributed computing

2.3 Deep Learning Architectures

2.3.1 Convolutional Neural Networks (CNNs)

CNNs are specialized neural networks designed for processing grid-like data such as images. They exploit spatial locality and translational invariance through learnable filters.

Core CNN components:

- Convolutional Layer: Applies learnable filters via cross-correlation. Output feature map size: $(W-F+2P)/S + 1$, where W=input size, F=filter size, P=padding, S=stride

- Pooling Layer: Downsamples feature maps to reduce spatial dimensions. Types: Max pooling (retains strongest activation), Average pooling
- Activation Functions: ReLU most common — ReLU(x) = max(0, x)
- Fully Connected Layers: Final classification/regression head

Landmark CNN architectures:

- LeNet-5 (1998): First successful CNN for digit recognition
- AlexNet (2012): Winner of ImageNet — popularized deep learning
- VGGNet (2014): Very deep networks with small 3×3 filters
- ResNet (2015): Introduced residual connections — enabled very deep networks (50-152+ layers)
- EfficientNet (2019): Compound scaling of depth, width, and resolution

2.3.2 Recurrent Neural Networks (RNNs)

RNNs process sequential data by maintaining a hidden state that captures information about previous inputs. The hidden state is updated at each time step using both the current input and the previous hidden state.

RNN Equations

ht = tanh(Wh * h_{t-1} + Wx * xt + b) — hidden state updateyt = Wy * ht + by — output at time tBackpropagation Through Time (BPTT): Unroll the network through time steps and apply standard backpropProblem: Vanishing/exploding gradients for long sequences

2.3.3 Long Short-Term Memory (LSTM)

LSTMs solve the vanishing gradient problem in RNNs by introducing a cell state (Ct) and three gating mechanisms that regulate information flow.

- Forget Gate: ft = σ(Wf * [h_{t-1}, xt] + bf) — decides what to discard from cell state
- Input Gate: it = σ(Wi * [h_{t-1}, xt] + bi) — decides what new info to store
- Cell Update: C̃t = tanh(WC * [h_{t-1}, xt] + bC) — candidate cell state values
- Cell State: Ct = ft * C_{t-1} + it * C̃t — updated cell state
- Output Gate: ot = σ(Wo * [h_{t-1}, xt] + bo) — output based on filtered cell state

2.3.4 Transformers

The Transformer architecture, introduced in 'Attention is All You Need' (Vaswani et al., 2017), revolutionized sequence modeling by relying entirely on attention mechanisms, eliminating recurrence.

Self-Attention Mechanism

Scaled Dot-Product Attention:Attention(Q, K, V) = softmax(QK^T / √dk) * VQ = Queries, K = Keys, V = Values, dk = dimension of keysMulti-Head Attention: Run h parallel attention heads and concatenate outputsAllows each position to attend to all other positions — captures long-range dependencies

Transformer key components:

- Positional Encoding: Injects position information since no recurrence — uses sine/cosine functions
- Multi-Head Attention: Multiple attention heads capture different relationship types
- Feed-Forward Layers: Position-wise FFN applied to each position independently
- Layer Normalization: Applied before or after sublayers (Pre-LN or Post-LN)
- Residual Connections: Skip connections around each sublayer

Notable Transformer models: BERT (bidirectional encoder), GPT series (autoregressive decoder), T5 (encoder-decoder), Vision Transformers/ViT (images), DALL-E, GPT-4, Claude.

2.4 Transfer Learning and Domain Adaptation

Transfer learning leverages knowledge gained from training on one task to improve performance on a related but different task. It is especially valuable when labeled data is scarce, as it allows models pretrained on large datasets to be adapted with minimal task-specific data.

2.4.1 Transfer Learning Strategies

- Feature Extraction: Use pretrained model as fixed feature extractor. Freeze all pretrained layers and train only a new classification head. Best when source and target domains are very similar.
- Fine-tuning: Unfreeze some or all pretrained layers and continue training on target task. Use a small learning rate to avoid catastrophic forgetting. Best for moderate domain shift.
- Full Fine-tuning: Unfreeze and train the entire network. Requires more target data. Used when source and target domains differ significantly.

2.4.2 Domain Adaptation

Domain adaptation addresses the scenario where the source domain (where the model was trained) and the target domain (where it will be deployed) have different data distributions — covariate shift, label shift, or concept drift.

- Covariate Shift: $P(x)$ differs between domains but $P(y|x)$ remains the same
- Label Shift: $P(y)$ differs — prevalence of classes changes
- Concept Drift: $P(y|x)$ changes — the relationship itself changes over time

Techniques: Instance reweighting, domain adversarial training (DANN), Maximum Mean Discrepancy (MMD) minimization, and self-training with pseudo-labels.

2.4.3 Pretrained Models and Foundation Models

Foundation models are large models pretrained on massive datasets that serve as a starting point for many downstream tasks. Examples include: BERT, GPT-4, LLaMA (NLP), ResNet, EfficientNet, ViT (vision), CLIP (vision-language), Whisper (speech).

2.5 Multi-task Learning and Meta-learning

2.5.1 Multi-task Learning (MTL)

Multi-task learning trains a single model to perform multiple related tasks simultaneously. By sharing representations across tasks, MTL enables the model to leverage information from all tasks, improving generalization and data efficiency.

MTL architectures:

- Hard Parameter Sharing: Share lower layers across all tasks; task-specific output heads. Most common and simplest MTL approach.
- Soft Parameter Sharing: Each task has its own parameters, but with constraints encouraging similarity (e.g., L2 regularization between corresponding parameters).
- Mixture of Experts (MoE): Shared expert networks with task-specific gating mechanisms.

Benefits of MTL: Acts as implicit regularization (reduces overfitting), enables learning from auxiliary tasks when main task has limited data, and can improve feature representations through task-specific inductive biases.

2.5.2 Meta-learning (Learning to Learn)

Meta-learning trains models to quickly adapt to new tasks with minimal data. The goal is to learn a learning algorithm rather than learning a specific task — essentially, learning how to learn.

Meta-learning Setup

Meta-training: Train across many tasks {T1, T2, ..., Tm} from a task distributionMeta-test: Evaluate adaptation on new, unseen tasksSupport Set (S): Few examples per class used for adaptation (k-shot)Query Set (Q): Examples used to evaluate after adaptation

2.5.3 MAML (Model-Agnostic Meta-Learning)

MAML finds model initialization parameters θ that can be quickly adapted to any new task with few gradient steps. The outer loop optimizes for fast adaptation; the inner loop performs task-specific adaptation.

- Inner loop: $\theta' = \theta - \alpha \nabla L_{task}(\theta)$ — task-specific adaptation
- Outer loop: $\theta \leftarrow \theta - \beta \nabla \Sigma L_{task}(\theta')$ — meta-parameter update
- Model-agnostic: Works with any gradient-based model
- Applications: Few-shot classification, robotics, neural architecture search

2.5.4 Prototypical Networks and Metric-based Meta-learning

Prototypical networks learn an embedding space where classification is performed by computing distances to class prototypes. The prototype for each class is the mean embedding of its support examples.

Other metric-based approaches include Siamese Networks (learn similarity functions), Matching Networks (attention over support set), and Relation Networks (learn a relation module for comparison).

QUESTION & ANSWER SECTION

This section contains 50 questions organized by mark value. Attempt all questions for comprehensive preparation.

— SECTION A: 10-MARK QUESTIONS —

Q1. Explain the bias-variance trade-off in machine learning. Discuss how overfitting and underfitting relate to bias and variance, and describe at least three strategies to find the optimal balance. [10 Marks]

Answer:
The bias-variance trade-off states that Total Error = Bias² + Variance + Irreducible Noise. Bias measures how far predictions are from true values on average, arising from incorrect assumptions (e.g., fitting a linear model to non-linear data). High bias causes underfitting — poor performance on both training and test sets. Variance measures sensitivity to fluctuations in training data. High variance causes overfitting — excellent training performance but poor test performance. Strategies to find balance: (1) Regularization (L1/L2, dropout) penalizes complexity and reduces variance. (2) Cross-validation helps detect overfitting and select optimal model complexity. (3) Ensemble methods like bagging reduce variance by averaging predictions across many models. (4) Early stopping halts training when validation error starts increasing. (5) Increasing training data reduces variance. The optimal model complexity is found where the sum of bias² and variance is minimized on the validation set.

Q2. Describe the architecture of a Transformer model in detail. Explain the self-attention mechanism, positional encoding, and multi-head attention with mathematical formulations. [10 Marks]

Answer:
The Transformer (Vaswani et al., 2017) consists of stacked encoder and decoder blocks. Encoder: N identical layers, each with (1) Multi-Head Self-Attention and (2) Position-wise Feed-Forward Network, with residual connections and layer normalization around each sublayer. Decoder: Same plus cross-attention between decoder queries and encoder outputs. Self-Attention: Q, K, V matrices are linear projections of inputs. Attention(Q,K,V) = softmax($QK^T/\sqrt{dk}$) * V, where scaling by $\sqrt{dk}$ prevents vanishing gradients in softmax. Multi-Head Attention: Runs h parallel attention heads: MultiHead(Q,K,V) = Concat(head1,...,headh)Wo, where headi = Attention(QWi_Q, KWi_K, VWi_V). This allows the model to jointly attend to information from different representation subspaces. Positional Encoding: Since attention is permutation-invariant, positional information is injected using PE(pos,2i) = sin(pos/10000^(2i/d)) and PE(pos,2i+1) = cos(pos/10000^(2i/d)). These are added to input embeddings. The Transformer enables parallel computation (unlike RNNs), captures long-range dependencies, and scales efficiently. It is the foundation of BERT, GPT, T5, and virtually all modern large language models.

Q3. Explain XGBoost in detail. Discuss its objective function, regularization, split-finding algorithm, and key advantages over traditional Gradient Boosting Machines. [10 Marks]

Answer:

XGBoost (Extreme Gradient Boosting) is an optimized gradient boosting framework. Objective Function: $L(\varphi) = \Sigma l(y_i, \hat{y}_i) + \Sigma[\gamma T + (1/2)\lambda \Sigma w_j^2]$, where T = number of leaves, w_j = leaf weights, γ = leaf penalty, λ = L2 regularization. Second-Order Optimization: XGBoost uses both gradient g and Hessian h of the loss: optimal weight for leaf j is $w_j^* = -G_j/(H_j + \lambda)$, where $G_j = \Sigma g_i$ and $H_j = \Sigma h_i$ for samples in leaf j. Optimal gain for a split: $\text{Gain} = (1/2)[G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - (G_L+G_R)^2/(H_L+H_R+\lambda)] - \gamma$. Key Advantages: (1) Regularization prevents overfitting via leaf penalties. (2) Second-order gradients provide more accurate optimization than GBM. (3) Sparsity-aware algorithm handles missing values by learning optimal default directions. (4) Tree pruning uses max_depth and prunes unprofitable splits. (5) Column and row subsampling add randomness like Random Forests. (6) Approximate quantile sketch enables distributed, approximate split finding. (7) Cache-aware access and out-of-core computation enable scalability. XGBoost consistently wins Kaggle competitions on structured/tabular data.

Q4. Describe ensemble learning methods: bagging, boosting, and stacking. Compare their mechanisms, strengths, weaknesses, and suitable use cases with examples. [10 Marks]

Answer:

Bagging (Bootstrap Aggregating): Trains B models on bootstrap samples (with replacement) of the training data. Final prediction: majority vote (classification) or mean (regression). Reduces variance without increasing bias. Models are independent — can be trained in parallel. Example: Random Forest (adds random feature selection per split). Strengths: Robust to outliers, parallelizable, free OOB error estimate. Weakness: Limited improvement if base models are correlated. Boosting: Trains models sequentially. Each model focuses on examples the previous ensemble misclassified. Examples: AdaBoost (reweights misclassified examples), Gradient Boosting (fits to negative gradient/residuals), XGBoost (adds regularization and second-order optimization). Strengths: Generally achieves highest accuracy on structured data, automatically handles feature interactions. Weakness: Sequential training — not parallelizable; sensitive to noisy data and outliers. Stacking: Level-0 learners (diverse algorithms — SVM, RF, XGBoost, Neural Net) make predictions. Level-1 meta-learner is trained on these predictions (using cross-validation to prevent leakage). Strengths: Can leverage complementary strengths of diverse models. Weakness: Complex to implement, computationally expensive, risk of overfitting. Use Cases: Bagging — high-dimensional data, when many features are relevant; Boosting — tabular data competitions, when accuracy is paramount; Stacking — competition settings where maximum performance is needed.

Q5. Explain the SVM optimization problem for both linearly and non-linearly separable cases. Describe the kernel trick and compare three commonly used kernels with their mathematical formulations. [10 Marks]

Answer:

Linear SVM (Hard Margin): For linearly separable data, find w, b minimizing $(1/2)||w||^2$ subject to $y_i(w^T x_i+b) \geq 1$. The margin is $2/||w||$. Support vectors are points satisfying $y_i(w^T x_i+b)=1$. Dual formulation via Lagrangians: maximize $\Sigma\alpha_i - (1/2)\Sigma\Sigma\alpha_i\alpha_j y_i y_j (x_i^T x_j)$, subject to $\alpha_i \geq 0$, $\Sigma\alpha_i y_i=0$. Soft Margin SVM: Introduces slack ξ_i for non-separable data. Minimize $(1/2)||w||^2 + C\Sigma\xi_i$, subject to $y_i(w^T x_i+b) \geq 1-\xi_i$, $\xi_i \geq 0$. C controls margin-error trade-off. The Kernel Trick: Replace $x_i^T x_j$ in the dual with $K(x_i,x_j)=\varphi(x_i)^T\varphi(x_j)$, enabling implicit mapping to high-dimensional feature spaces. Kernels: (1) Linear: $K(x_i,x_j) = x_i^T x_j$ — for linearly separable data, fast; (2) Polynomial: $K(x_i,x_j) = (x_i^T x_j + c)^d$ — captures feature interactions up to degree d, hyperparameters c and d; (3) RBF/Gaussian: $K(x_i,x_j) = \exp(-\gamma||x_i-x_j||^2)$ — infinite-dimensional feature space, universal

approximator, most popular; sensitive to γ (bandwidth). γ too large → overfitting; too small → underfitting. SVMs are effective in high-dimensional spaces, memory efficient (only support vectors matter), and robust with proper kernels. Main disadvantages: computationally expensive for large n, sensitive to feature scaling and hyperparameter choice.

Q6. Discuss regularization techniques in deep learning: L1/L2 regularization, Dropout, and Batch Normalization. Explain the mathematical formulation and the mechanism by which each prevents overfitting. **[10 Marks]**

Answer:

L1 Regularization (Lasso): Adds λΣ|wj| to the loss. Gradient: ∂L/∂wj += λ*sign(wj). Promotes exact sparsity — many weights become zero — performing implicit feature selection. Geometrically, the L1 ball has corners along axes, making sparse solutions likely. L2 Regularization (Ridge): Adds (λ/2)Σwj² to the loss. Gradient: ∂L/∂wj += λwj (weight decay). Drives weights toward zero but rarely to exactly zero. Equivalent to assuming Gaussian prior on weights. Handles correlated features by distributing weights. Elastic Net combines both: αL1 + (1-α)L2 penalty. Dropout: During training, each neuron is independently set to 0 with probability p. Network learns with random subsets of neurons, preventing co-adaptation. At inference, multiply weights by (1-p) or use inverted dropout. Mathematically equivalent to training 2^n model ensemble. Typical p = 0.2-0.5. DropConnect is a generalization that drops connections (weights) rather than neurons. Batch Normalization: For mini-batch B, normalizes activations: x̂i = (xi - μB)/√(σ²B + ε), then applies learnable γ,β: yi = γx̂i + β. Benefits: (1) Reduces internal covariate shift, enabling higher learning rates. (2) Provides regularization effect — acts like adding noise (batch statistics vary). (3) Reduces sensitivity to weight initialization. (4) Can reduce need for Dropout. Variants: Layer Norm (over features, not batch — better for RNNs/Transformers), Group Norm, Instance Norm, all addressing batch-size dependence.

Q7. Explain transfer learning and domain adaptation. Describe fine-tuning strategies, when each should be used, and discuss the challenges of catastrophic forgetting with techniques to mitigate it. **[10 Marks]**

Answer:

Transfer learning transfers knowledge from a source domain/task to a target domain/task, leveraging pretrained model representations. Three main strategies: (1) Feature Extraction: Freeze all pretrained layers (no gradient flow), add a new trainable head. Only trains on target data. Best when: target dataset is small and similar to source. Avoids catastrophic forgetting entirely. (2) Partial Fine-tuning: Freeze early layers (low-level features), unfreeze later layers and head. Use small learning rate (e.g., 1e-4 vs 1e-3 for head). Early layers capture universal features (edges, textures); later layers are task-specific. (3) Full Fine-tuning: Unfreeze all layers. Requires more target data. Risk of catastrophic forgetting if learning rate is too large. Domain Adaptation: Addresses distribution shift between source and target domains. Unsupervised DA (no target labels): Adversarial methods (DANN) train a domain discriminator adversarially to learn domain-invariant features. MMD minimization aligns feature distributions. Semi-supervised DA: Uses few labeled target examples. Catastrophic Forgetting: When fine-tuning, the model overwrites pretrained weights, losing prior knowledge. Mitigation techniques: (1) Elastic Weight Consolidation (EWC): Penalizes changes to weights important for previous tasks (weighted by Fisher information). (2) Progressive Neural Networks: Freeze old columns, add new lateral connections. (3) Learning rate scheduling: Very small LR for pretrained layers. (4) Gradual unfreezing: Unfreeze layers from top to bottom progressively. (5) Replay methods: Mix old task examples during training. Transfer learning significantly reduces training time and data

requirements — ResNet pretrained on ImageNet can achieve excellent performance on medical imaging tasks with only hundreds of labeled examples.

Q8. Describe the architecture of LSTM and GRU. Explain how they solve the vanishing gradient problem in vanilla RNNs, and compare their mechanisms, computational complexity, and suitable applications. **[10 Marks]**

Answer:

Vanilla RNN Problem: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$. During BPTT, gradients are multiplied by W_h at each time step. If spectral radius < 1, gradients vanish exponentially; if > 1, they explode. This makes learning long-term dependencies impossible. LSTM Solution — 4 Gates: Forget Gate: $f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$ — what fraction of cell state to retain. Input Gate: $i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$ — what new info to write. Cell Candidate: $\tilde{C}_t = \tanh(W_C [h_{t-1}, x_t] + b_C)$ — candidate values. Cell Update: $C_t = f_t C_{t-1} + i_t \tilde{C}_t$ — key: additive update maintains gradient flow. Output Gate: $o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$; $h_t = o_t \tanh(C_t)$. The cell state C_t acts as a conveyor belt — gradients flow through the additive path without multiplicative decay (when $f_t \approx 1$). GRU (Gated Recurrent Unit): Simplified with 2 gates. Reset Gate: $r_t = \sigma(W_r [h_{t-1}, x_t])$ — how much past to forget. Update Gate: $z_t = \sigma(W_z [h_{t-1}, x_t])$ — balance between old and new state. $\tilde{h}_t = \tanh(W [r_t h_{t-1}, x_t])$; $h_t = (1-z_t) h_{t-1} + z_t \tilde{h}_t$. Comparison: LSTM has separate cell state + hidden state, 4 weight matrices per layer, better for very long sequences, more parameters. GRU merges cell/hidden state, 3 weight matrices, faster to train, comparable performance for moderate sequence lengths. Applications: LSTM — language modeling, machine translation, speech recognition, time series. GRU — when computational resources limited, streaming applications. Both outperformed by Transformers for most NLP tasks today but still used for time series and streaming data.

Q9. Explain meta-learning (learning to learn). Describe the MAML algorithm in detail with mathematical formulation. Compare MAML with prototypical networks and discuss the N-way K-shot classification setup. **[10 Marks]**

Answer:

Meta-learning trains models that can quickly adapt to new tasks from few examples. The goal: find model parameters θ such that a few gradient steps on a new task yield good performance. N-way K-shot Classification Setup: Each task T_i has N classes, K labeled support examples per class (support set S), and Q query examples (query set Q). Model must classify query examples correctly after seeing only K examples per class. Typical settings: 5-way 1-shot, 5-way 5-shot, 20-way 1-shot. MAML (Model-Agnostic Meta-Learning): Goal: Find θ such that $\theta' = \theta - \alpha \nabla L_{T_i}(\theta)$ (inner loop adaptation) produces good performance on T_i. Outer loop: $\theta \leftarrow \theta - \beta \nabla \Sigma_i L_{T_i}(\theta'_i)$. The meta-gradient involves second-order derivatives (∇^2). Full MAML is expensive; First-Order MAML (FOMAML) and Reptile approximate it without second-order terms. Intuition: MAML finds a parameter initialization in a region of the loss landscape that has many good task-specific solutions nearby. Prototypical Networks: Learn embedding function $f_\varphi(x)$. Class prototype: $c_k = (1/K)\Sigma f_\varphi(x_i)$ for support examples of class k. Classification: $P(y=k|x) = \text{softmax}(-d(f_\varphi(x), c_k))$, where d is Euclidean distance. Training: episodic training mimics test-time N-way K-shot setup. Simpler, faster to train than MAML (no second-order gradients). Comparison: MAML is gradient-based — adaptable to any task; requires second-order optimization; can overfit with very few tasks. Prototypical Nets are metric-based — only need one forward pass to classify; less flexible for non-classification tasks; deterministic given support set. Both require episodic/meta-training from a large collection of source tasks.

Q10. Discuss cross-validation strategies in machine learning. Explain K-fold, Stratified K-fold, LOOCV, and Time Series cross-validation. How is cross-validation used for hyperparameter tuning and model selection? [10 Marks]

Answer:

Cross-validation (CV) assesses model generalization by evaluating on multiple data subsets, providing a more reliable estimate than a single train-test split. K-Fold CV: Dataset split into K equal folds. Train on K-1 folds, validate on held-out fold. Repeat K times. Final estimate = mean ± std of K scores. K=5 or K=10 standard — balances bias-variance in the CV estimate itself. Stratified K-Fold: Each fold maintains the original class distribution. Critical for imbalanced datasets (e.g., 95%/5% binary classification). Prevents a fold from having no positive examples. LOOCV (Leave-One-Out): K = n (one fold per sample). Nearly unbiased estimate but computationally O(n) model trainings. High variance in the CV estimate. Preferred for very small datasets (n < 50). Time Series CV: Standard K-fold shuffles data — invalid for time series (uses future data to predict past). Walk-forward validation: Train on [1..t], validate on [t+1..t+h]. Expanding window or sliding window variants. Purging and embargo techniques prevent data leakage near fold boundaries. Hyperparameter Tuning: Nested CV: Outer loop for unbiased performance estimate; inner loop for hyperparameter selection. Grid Search: Evaluate all combinations in a parameter grid — O(∏|Hi|) evaluations. Random Search: Sample parameter combinations randomly — more efficient than grid search for high-dimensional spaces (Bergstra & Bengio, 2012). Bayesian Optimization: Model the validation score as a function of hyperparameters using a Gaussian Process. Use acquisition function (EI, UCB) to select next hyperparameters intelligently. Significantly more efficient than grid/random search. Hyperband: Combines early stopping with random search — allocates more resources to promising configurations. Model Selection: Use CV to choose between different model architectures, feature sets, or algorithms. Report final performance on a completely held-out test set (never used in model selection) to avoid optimistic bias.

Q11. Explain CNN architectures in detail. Describe convolutional layers, pooling, and modern architectures like ResNet. How do residual connections solve the degradation problem and enable very deep networks? [10 Marks]

Answer:

Convolutional Neural Networks exploit three key ideas: local connectivity, weight sharing (same filter applied across all positions), and hierarchical feature learning. Convolutional Layer: For input I and filter K of size f×f, feature map S[i,j] = Σ_m Σ_n I[i+m, j+n] * K[m,n]. Output size: (W-F+2P)/S + 1. Each filter detects a specific pattern (edge, texture, part). Multiple filters → depth dimension. Learnable parameters per layer: F×F×C_in×C_out + C_out (biases). Pooling Layer: Max pooling: takes maximum in each f×f window, stride s. Reduces spatial dimensions by s, C unchanged. Provides translational invariance and reduces computation. Average pooling: used in global average pooling (GAP) for classification head in modern CNNs. Activation: ReLU(x) = max(0,x) — prevents vanishing gradients, computationally efficient. ResNet (He et al., 2015): Degradation Problem: Adding more layers to a network beyond a certain depth causes accuracy to saturate and then degrade — not due to overfitting but optimization difficulty. Residual Learning: Instead of learning H(x), learn the residual F(x) = H(x) - x. Output: H(x) = F(x) + x (skip/shortcut connection). If optimal solution is close to identity, it's easier to push F(x) → 0 than to learn H(x) → x from scratch. Gradient flow: ∂L/∂x = ∂L/∂H * (∂F/∂x + 1). The '+1' term ensures gradients flow directly without vanishing, even for very deep networks (50-152+ layers). Variants: Bottleneck blocks (1×1, 3×3, 1×1 conv) for parameter efficiency; ResNeXt adds grouped convolutions; DenseNet connects every layer to all subsequent layers; EfficientNet uses compound scaling.

Q12. Describe multi-task learning (MTL) in neural networks. Explain hard and soft parameter sharing, the challenges of task weighting, and provide examples of successful MTL applications. [10 Marks]

Answer:

Multi-task learning (MTL) trains a single model to simultaneously solve multiple related tasks, sharing representations across tasks. This acts as a form of inductive bias, improving generalization through auxiliary task supervision. Hard Parameter Sharing: Share all hidden layers across tasks; each task has its own output head. Joint loss: $L = \Sigma_t w_t * L_t$, where w_t are task weights. Architecture: input → shared encoder (multiple layers) → task-specific heads (linear, MLP). Advantages: Reduces parameters dramatically, acts as regularization (model cannot overfit to one task's noise), enables learning from tasks with little labeled data. Risk: Negative transfer if tasks conflict — one task's gradient may hurt others. Soft Parameter Sharing: Each task has its own parameters θ_t. Regularization forces parameters close across tasks: $L_{reg} = \lambda\Sigma_{t1\neq t2}||\theta_{t1} - \theta_{t2}||^2$. More flexible but more parameters. Variants: Cross-stitch Networks (learn task-specific linear combinations of activations), Sluice Networks. Task Weighting Challenges: Fixed weights may under/over-weight tasks. Solutions: (1) Uncertainty weighting (Kendall et al.): $w_t = 1/(2\sigma_t^2)$, where σ_t^2 is learned per-task uncertainty — automatically balances. (2) GradNorm: Normalizes gradient magnitudes across tasks to maintain balance. (3) PCGrad: Projects conflicting gradients to prevent interference. Successful Applications: (1) NLP: BERT jointly trained on MLM + NSP; MT-DNN on multiple NLP benchmarks. (2) Computer Vision: Object detection + semantic segmentation + depth estimation jointly. (3) Autonomous Driving: Jointly predicting steering, braking, acceleration. (4) Drug Discovery: Predicting multiple molecular properties simultaneously. (5) Recommendation Systems: Jointly optimizing clicks, purchases, watch time.

Q13. What is Random Forest? Explain the algorithm, hyperparameters, feature importance, and how it improves upon single decision trees. Discuss its strengths and limitations. [10 Marks]

Answer:

Random Forest (Breiman, 2001) is an ensemble of decision trees trained using bagging and random feature selection. Algorithm: (1) For b = 1 to B: sample bootstrap dataset D_b (n samples with replacement). (2) Train a decision tree T_b on D_b with one modification: at each split, consider only m random features ($m \approx \sqrt{p}$ for classification, p/3 for regression) instead of all p features. (3) Aggregate: majority vote (classification) or mean (regression) across all B trees. Key Hyperparameters: n_estimators: Number of trees — more is better up to diminishing returns (typically 100-500). max_features: Features considered per split (key randomization parameter). max_depth: Limits tree depth — prevents individual trees from overfitting. min_samples_split/leaf: Minimum samples required to split or be a leaf — controls tree complexity. bootstrap: Whether to use bootstrap sampling (True by default). OOB Score: ~37% of samples are out-of-bag for each tree. These are used to compute OOB error — a free validation estimate without needing a held-out set. Feature Importance: For each feature, sum the weighted impurity decrease (Gini/entropy) across all splits using that feature, averaged over all trees. Permutation importance: shuffle a feature's values and measure decrease in OOB accuracy — more reliable, accounts for correlations. Improvements over Single Trees: (1) Much lower variance due to averaging; (2) Robust to outliers; (3) Handles missing data; (4) No feature scaling needed; (5) Implicit feature selection via importance. Limitations: (1) Less interpretable than a single tree; (2) Slower prediction (O(B×depth) per prediction); (3) Poor performance on high-cardinality categorical features without encoding; (4) May not capture certain sequential/interactive patterns as well as boosting.

Q14. Explain the concept of attention mechanisms. Compare self-attention, cross-attention, and multi-head attention. How does attention enable transformers to handle long-range dependencies better than RNNs? [10 Marks]

Answer:

Attention mechanisms allow models to focus on relevant parts of the input when generating each output, rather than relying on a fixed-size context vector. Core Attention: Given a query Q, keys K, and values V, attention computes a weighted sum of values, where weights depend on query-key similarity. Scaled Dot-Product Attention: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$. Scaling by $\sqrt{d_k}$: prevents softmax from operating in saturation regions (small gradients) when d_k is large (dot products grow large). Self-Attention: Q, K, V all come from the same sequence. Each position attends to all other positions. Enables capturing global dependencies in O(1) path length between any two positions. In an encoder, each token builds a contextual representation attending to the entire input. Cross-Attention: Q comes from decoder/target sequence; K, V come from encoder output. Allows decoder to attend to relevant encoder positions (like alignment in translation). Multi-Head Attention: h parallel attention operations with different learned projections: $\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_i,...,\text{head}_h)W^O$, where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$. Different heads learn to attend to different aspects (syntactic, semantic, coreference, etc.). Attention vs RNNs for Long-Range Dependencies: RNNs: Information must pass through sequential hidden state updates. For a dependency between positions i and j (distance $d=|i-j|$), gradients must backpropagate through d steps — vanishing gradients for large d. Even LSTMs struggle with very long-range dependencies. Self-Attention: Every pair of positions has a direct connection — path length = O(1). No sequential bottleneck. Memory complexity $O(n^2)$ per layer — computationally expensive for very long sequences ($n > 10k$). Solutions: sparse attention (Longformer), local-global attention (BigBird), linear attention approximations. The O(1) path length and parallel computation are the key advantages that made Transformers dominate NLP.

Q15. What are the key differences between supervised, unsupervised, and reinforcement learning? Provide a comprehensive comparison of their problem formulations, algorithms, evaluation metrics, and real-world applications. [10 Marks]

Answer:

Supervised Learning: Problem: Given labeled dataset $\{(x_i, y_i)\}$, learn $f: X \rightarrow Y$. Training Signal: Direct supervision — ground truth labels. Goal: Minimize empirical risk $L = (1/n)\Sigma l(f(x_i), y_i)$. Algorithms: Linear/Logistic Regression, Decision Trees, Random Forest, SVM, Neural Networks, Transformers. Evaluation: Accuracy, Precision/Recall/F1 (classification); MSE, MAE, R^2 (regression). Applications: Email spam detection, image classification, medical diagnosis, machine translation, speech recognition. Key Challenge: Requires large labeled datasets (expensive to annotate). Unsupervised Learning: Problem: Given unlabeled dataset $\{x_i\}$, discover structure $P(x)$. Training Signal: No labels — learn from data distribution itself. Goal: Compress, cluster, generate, or understand data. Algorithms: K-means, DBSCAN (clustering); PCA, t-SNE, UMAP (dim reduction); VAE, GAN (generative); ICA, NMF (decomposition). Evaluation: Silhouette score, Davies-Bouldin (clustering); FID (generative models); reconstruction error (autoencoders). Applications: Customer segmentation, anomaly detection, data compression, drug discovery, representation learning. Key Challenge: No ground truth — difficult to evaluate quality objectively. Reinforcement Learning: Problem: Agent interacts with environment (MDP: states S, actions A, rewards R, transition P, discount γ). Goal: Find policy π^* maximizing expected cumulative reward $E[\Sigma\gamma^t r_t]$. Algorithms: Q-Learning, DQN (value-based); REINFORCE, PPO, A3C (policy gradient); TD3, SAC (actor-critic). Evaluation: Cumulative reward, sample efficiency, policy convergence. Applications: Game playing (AlphaGo, Atari), robotics, autonomous vehicles, algorithmic trading, drug dosing optimization. Key Challenge: Sparse rewards, exploration-exploitation dilemma, credit assignment over long horizons.

Connections: Self-supervised learning (masked autoencoding) bridges supervised and unsupervised — generates supervision from data itself. Reward shaping in RL can use supervised learning for reward models (RLHF).

— SECTION B: 5-MARK QUESTIONS —

Q16. What is the difference between L1 and L2 regularization? When would you prefer L1 over L2? [5 Marks]

Answer:
L1 (Lasso) adds $\lambda\Sigma|w_j|$ to the loss, producing sparse solutions (many weights = 0) — useful for feature selection. Gradient: λ*sign(w_j). L2 (Ridge) adds $(\lambda/2)\Sigma w_j^2$, producing small but nonzero weights. Gradient: λw_j (weight decay). Key difference: L1 creates sparsity geometrically (L1 ball has corners on axes), L2 creates uniform shrinkage. Prefer L1 when: (1) You believe few features are truly relevant (sparse ground truth), (2) You want implicit feature selection, (3) Interpretability is important (only nonzero features matter). Prefer L2 when: (1) All features contribute somewhat, (2) Features are correlated (L1 picks one arbitrarily), (3) You need stable, convex optimization. Elastic Net combines both: $\alpha L1 + (1-\alpha)L2$, useful when many correlated features exist.

Q17. Explain the concept of bootstrapping and how it is used in Random Forests. [5 Marks]

Answer:
Bootstrapping is a statistical resampling technique that creates new datasets by sampling with replacement from the original data. A bootstrap sample of size n draws n examples from the original n examples with replacement — approximately 63.2% unique samples, 36.8% are duplicates. In Random Forests: (1) For each tree, create a bootstrap sample from the training data. (2) Train a full decision tree on this sample. (3) The ~36.8% unsampled examples (out-of-bag/OOB samples) are used for free validation. OOB Error: For each training example, average predictions from trees that did NOT see it. Provides an unbiased generalization estimate without needing a separate validation set. The randomness from bootstrap sampling, combined with random feature selection, ensures tree diversity — diverse trees that make different errors produce better ensembles through averaging (variance reduction without bias increase).

Q18. Describe the vanishing gradient problem in deep neural networks and list techniques to address it. [5 Marks]

Answer:
Vanishing gradients occur when backpropagation multiplies gradients through many layers, and if these are < 1 (e.g., from sigmoid/tanh activations), gradients shrink exponentially. For a 100-layer network with sigmoid, gradients can become $10^{(-30)}$, making lower layers learn nearly nothing. Causes: Saturating activations (sigmoid, tanh flatten for large inputs), poor weight initialization, very deep architectures. Solutions: (1) ReLU and variants (Leaky ReLU, ELU, GELU): Non-saturating activations pass gradients cleanly. (2) Batch Normalization: Normalizes activations, keeps them in good gradient regime. (3) Residual Connections (ResNets): Direct gradient paths bypass layers. (4) Xavier/He Initialization: Carefully scales initial weights to maintain variance. (5) Gradient Clipping: Clips gradient norm to prevent explosion. (6) LSTM/GRU: For RNNs, gating mechanisms allow selective gradient flow. (7) Careful architecture design: Shorter paths, dense connections (DenseNet).

Q19. What is the kernel trick in SVMs? Explain why it is computationally advantageous. [5 Marks]

Answer:

The kernel trick allows SVMs to compute similarities in a high-dimensional (possibly infinite-dimensional) feature space $\varphi(x)$ without explicitly computing $\varphi(x)$. Instead, kernel function $K(x_i, x_j) = \varphi(x_i)^T\varphi(x_j)$ computes the inner product directly. SVM dual optimization: $\max \Sigma\alpha_i - (1/2)\Sigma\Sigma\alpha_i\alpha_j y_i y_j K(x_i,x_j)$. Only $K(x_i,x_j)$ appears — never $\varphi(x)$ directly. Computational Advantage: Computing $\varphi(x)$ for RBF kernel would require an infinite-dimensional vector. Computing $K(x_i,x_j) = \exp(-\gamma||x_i-x_j||^2)$ is $O(d)$ — the same as computing the original distance. Example: Polynomial kernel $K(x_i,x_j) = (x_i^T x_j+c)^d$ computes interactions of all degree-d features without enumerating $O(d^d)$ features. Classification rule: $f(x) = \Sigma \alpha_i y_i K(x_i,x) + b$ — only support vectors ($\alpha_i > 0$) matter. This is called the representer theorem.

Q20. What is batch normalization? Explain its mathematical formulation and the role of learnable parameters γ and β. [5 Marks]

Answer:

Batch Normalization (Ioffe & Szegedy, 2015) normalizes activations within a mini-batch to stabilize training. For mini-batch $B = \{x_1,...,x_m\}$: (1) Batch mean: $\mu_B = (1/m)\Sigma x_i$; (2) Batch variance: $\sigma^2_B = (1/m)\Sigma(x_i-\mu_B)^2$; (3) Normalize: $\hat{x}_i = (x_i-\mu_B)/\sqrt{(\sigma^2_B+\varepsilon)}$, where $\varepsilon \approx 1e-5$ prevents division by zero; (4) Scale and shift: $y_i = \gamma\hat{x}_i + \beta$. γ (scale) and β (shift) are learnable parameters (one per feature). They allow the network to undo normalization if needed — without them, the network is constrained to zero-mean unit-variance activations which may not be optimal. At inference: use running estimates of mean and variance computed during training (exponential moving average). Benefits: (1) Enables higher learning rates; (2) Less sensitive to initialization; (3) Acts as regularizer (slight noise from batch statistics); (4) Reduces need for Dropout. Limitation: Sensitive to batch size — doesn't work well with very small batches; replaced by Layer Norm in Transformers.

Q21. Explain the difference between bagging and boosting. Which reduces bias and which reduces variance? [5 Marks]

Answer:

Bagging (Bootstrap Aggregating): Trains B models independently on bootstrap samples. Combines via averaging/voting. Reduces Variance: Each model has similar bias to a single model. Averaging independent noisy models cancels out errors — variance decreases roughly by 1/B (if models are uncorrelated). Bias stays the same. Best for high-variance, low-bias base models (like deep decision trees). Example: Random Forest. Boosting: Trains models sequentially, each correcting previous errors. Reduces Bias: Each iteration adds a model that addresses the remaining error (residuals/misclassified examples). Each base model is weak (high bias), but the ensemble becomes increasingly accurate. Variance can increase with too many boosting rounds (overfitting). Best with weak learners (shallow trees, depth 1-5). Example: AdaBoost, XGBoost. Summary: Bagging = high-variance models → reduces variance (averaging). Boosting = high-bias models → reduces bias (sequential error correction). In practice, boosting often achieves lower test error but is more prone to overfitting and sensitive to noisy data. Bagging is more robust and parallelizable.

Q22. Describe the MAML algorithm. What is the key idea behind it and what makes it model-agnostic? [5 Marks]

Answer:

MAML (Model-Agnostic Meta-Learning, Finn et al., 2017) learns an initialization θ that can be quickly adapted to new tasks with few gradient steps. Key Idea: Instead of learning task-specific parameters, find θ* such that after a few gradient steps on any new task Ti, the model performs well on Ti. Algorithm: Sample a batch of tasks {T1,...,Tk}. For each Ti: compute task-specific parameters: $\theta'_i = \theta - \alpha \nabla_\theta L_{T_i}(\theta)$ (inner loop, one or few gradient steps on support set). Meta-update: $\theta \leftarrow \theta - \beta \nabla_\theta \sum_i L_{T_i}(\theta'_i)$ (outer loop using query set loss). The meta-gradient $\nabla_\theta \sum_i L_{T_i}(\theta'_i)$ requires second-order derivatives — differentiating through the inner loop. FOMAML ignores second-order terms (cheaper, nearly as effective). Model-Agnostic: Works with any model trained with gradient descent — MLPs, CNNs, RNNs. No architectural changes needed; just requires differentiability. This enables application to classification, regression, and reinforcement learning tasks using the same algorithm.

Q23. What is stacking in ensemble learning? How is it different from voting and averaging? [5 Marks]

Answer:

Stacking (Stacked Generalization, Wolpert 1992) trains a meta-learner on base model predictions. Architecture: Level-0 (base) learners: Diverse algorithms (RF, XGBoost, SVM, Neural Net) trained on training data. Their predictions become new features. Level-1 (meta-learner): Typically simple model (logistic regression, linear regression, XGBoost) trained on base model predictions. Training Protocol: Use K-fold CV to generate out-of-fold (OOF) predictions for training the meta-learner. Prevents overfitting — base models never predict on their own training data. Differences from Voting/Averaging: Simple Voting/Averaging: Equal or weighted combination of base model predictions — static, no learning involved. Stacking: Meta-learner learns the optimal combination — can learn 'model A is better for samples of type X, model B for type Y.' More expressive — can capture non-linear combination strategies. Computationally more expensive (requires CV for OOF predictions) but typically achieves higher accuracy. Used extensively in Kaggle competitions. Can be extended to multiple stacking levels (though rarely beneficial in practice).

Q24. Explain the concept of transfer learning. What are the different strategies for fine-tuning pretrained models? [5 Marks]

Answer:

Transfer learning uses knowledge from a source task (with abundant data) to improve performance on a target task (with limited data). Pretrained models have learned general representations (edges, textures, shapes for images; grammar, semantics for text) transferable across tasks. Fine-tuning Strategies: (1) Feature Extraction (Frozen Base): Freeze all pretrained layers, train only a new task-specific head. Fast, prevents overfitting on small datasets. Best when source ≈ target domain. (2) Partial Fine-tuning: Freeze early layers (universal low-level features), unfreeze later layers and head with small LR (e.g., 1e-5 vs 1e-3). Adapts high-level representations while preserving low-level features. (3) Full Fine-tuning: Unfreeze all layers. Needs larger target dataset. Use discriminative learning rates — smaller LR for lower layers, larger for top layers. (4) Adapter Layers: Insert small trainable modules between frozen pretrained layers. Only train adapters — parameter efficient (only 1-3% new parameters). (5) LoRA (Low-Rank Adaptation): Decomposes weight updates into low-rank matrices — extremely parameter efficient for LLMs. Practical Rules: Dataset size and domain similarity determine

strategy. Small+similar: feature extraction. Small+different: problem is hard, try early layers only. Large+similar: full fine-tune. Large+different: full fine-tune from scratch or pretrained init.

Q25. What is the difference between hard and soft parameter sharing in multi-task learning? [5 Marks]

Answer:

Hard Parameter Sharing: A single shared network (encoder) is used for all tasks. Task-specific output heads are added on top. All tasks share the same hidden representation. Architecture: Input → Shared Layers → [Task 1 Head, Task 2 Head, ..., Task N Head]. Advantages: Strong regularization (model cannot overfit any single task), fewer total parameters, simple implementation. Disadvantage: Risk of negative transfer — conflicting task gradients can hurt each other. Best when tasks are highly related. Soft Parameter Sharing: Each task has its own network with its own parameters θ_t. A regularization term encourages parameters to stay close: $L_{reg} = \Sigma||\theta_i - \theta_j||^2$. Each task can learn a distinct representation while sharing structural knowledge. Advantages: More flexible, less risk of negative transfer. Disadvantages: More parameters (O(T) × single model), weaker regularization. Variants: Cross-stitch Networks learn task-specific combinations of shared activations. Sluice Networks generalize cross-stitch with learnable selective sharing. Mixture of Experts routes inputs to different expert networks based on task/input — combines benefits of both. Practical Guidance: Start with hard sharing for related tasks. Use soft sharing or MoE when tasks have different input characteristics or objectives.

Q26. What is gradient descent? Compare batch gradient descent, mini-batch gradient descent, and stochastic gradient descent. [5 Marks]

Answer:

Gradient descent minimizes loss $L(\theta)$ by updating parameters in the direction of the negative gradient: $\theta \leftarrow \theta - \eta\nabla L(\theta)$. Batch Gradient Descent (BGD): Uses the entire training dataset to compute gradient per update. Update: $\theta \leftarrow \theta - \eta * (1/n)\Sigma\nabla L(x_i, y_i; \theta)$. Advantages: Stable convergence, true gradient direction. Disadvantages: Very slow for large datasets, cannot update until all data processed, cannot use online learning. Stochastic Gradient Descent (SGD): Uses one random example per update. $\theta \leftarrow \theta - \eta\nabla L(x_i, y_i; \theta)$. Advantages: Very fast updates, can escape local minima (noisy updates), online learning possible. Disadvantages: High variance in gradient estimate, oscillates around minimum, noisy convergence. Mini-batch Gradient Descent: Uses a mini-batch of B examples (typically 32-512). $\theta \leftarrow \theta - \eta*(1/B)\Sigma_B \nabla L(x_i, y_i; \theta)$. Best of both worlds: reasonable variance estimate, vectorized computation (GPU efficient), frequent updates. Standard practice in deep learning. Practical Notes: Learning rate scheduling (step decay, cosine annealing) crucial for convergence. Advanced optimizers (Adam, RMSProp, AdaGrad) adapt learning rates per-parameter using gradient history.

Q27. Explain convolutional layers. What do the hyperparameters kernel size, stride, and padding do? [5 Marks]

Answer:

A convolutional layer applies learnable filters (kernels) to an input feature map to detect patterns. For input I ($H \times W \times C_{in}$) and filters K ($f \times f \times C_{in} \times C_{out}$), the j-th feature map: $S_j[i,k] = \Sigma_m \Sigma_n \Sigma_c I[i*s+m, k*s+n, c] * K[m,n,c,j] + b_j$. Kernel Size (f): Determines the receptive field — how large a region of the input each output sees. Larger kernels (7×7, 11×11): capture coarser, larger

patterns; more parameters. Smaller kernels (3×3): capture local, fine-grained patterns; VGGNet showed two 3×3 convolutions approximate one 5×5 with fewer parameters and more nonlinearity. Modern trend: 3×3 kernels dominate. Stride (s): Step size of filter as it slides across the input. s=1: output size ≈ input size. s=2: output spatial dimensions halved — replaces pooling in some architectures (e.g., strided convolutions). Larger stride → more downsampling, fewer computations. Output size (one dimension): floor((W - f + 2P)/s) + 1. Padding (P): Adds zeros around the input border. Same/zero padding (P=f//2 with s=1): output same size as input — preserves spatial dimensions. Valid padding (P=0): output shrinks by f-1 per dimension — border pixels processed fewer times. Padding prevents shrinking and ensures border pixels receive equal treatment.

Q28. What is the purpose of activation functions? Compare ReLU, Sigmoid, Tanh, and GELU. [5 Marks]

Answer:

Activation functions introduce nonlinearity into neural networks, enabling them to learn complex, non-linear mappings. Without them, deep networks collapse to a single linear transformation. Sigmoid: σ(x) = 1/(1+e^(-x)), output ∈ (0,1). Problems: Saturates for |x|>5 → vanishing gradients; not zero-centered (all activations positive → all-positive or all-negative gradients → slow convergence). Use: Output layer for binary classification only. Tanh: tanh(x) = (e^x - e^(-x))/(e^x + e^(-x)), output ∈ (-1,1). Zero-centered (better than sigmoid), but still saturates. Use: RNN hidden states. ReLU: f(x) = max(0,x). Advantages: No saturation for positive inputs, computationally trivial, fast convergence. Problem: Dying ReLU — neurons that receive large negative inputs always output 0 and their gradients are always 0 → they never update. Fix: Leaky ReLU (αx for x<0), Parametric ReLU (learned α), ELU. GELU (Gaussian Error Linear Unit): GELU(x) = x*Φ(x) ≈ x*sigmoid(1.702x). Smooth approximation — differentiable everywhere. Standard in BERT, GPT, and most modern Transformers. Better empirical performance than ReLU on NLP tasks. Stochastic interpretation: randomly zeros inputs based on their value. Recommendations: Hidden layers in CNNs/MLPs → ReLU or Leaky ReLU. Transformer hidden layers → GELU or SiLU. Recurrent units → tanh (bounded, zero-centered). Output: sigmoid (binary), softmax (multiclass), linear (regression).

Q29. What is domain adaptation? Explain covariate shift and how it can be handled. [5 Marks]

Answer:

Domain adaptation addresses the problem where a model trained on a source domain must generalize to a different but related target domain with a different data distribution. Types of Distribution Shift: Covariate Shift: P_s(x) ≠ P_t(x) but P(y|x) remains the same. Input distribution changes, but the labeling function is unchanged. Example: Classifier trained on daytime images (source) deployed on nighttime images (target) — same objects, different visual statistics. Label Shift (Prior Shift): P_s(y) ≠ P_t(y) but P(x|y) unchanged. Class frequencies change. Example: Disease prevalence changes across hospitals. Concept Drift: P_s(y|x) ≠ P_t(y|x) — the relationship itself changes. Handling Covariate Shift: (1) Importance Weighting (IWERM): Weight source samples by w(x) = P_t(x)/P_s(x). Minimizing weighted loss = target risk. Estimate ratio using density estimation or discriminator. (2) Domain Adversarial Training (DANN): Train a feature extractor that fools a domain discriminator while predicting labels well. Minimax objective: encoder learns domain-invariant features. (3) Maximum Mean Discrepancy (MMD): Minimize the distance between source and target feature distributions in RKHS. (4) Normalization: Batch/Layer normalization can partially reduce distribution mismatch. (5) Test-Time Adaptation: Fine-tune on target unlabeled data using entropy minimization or consistency regularization.

Practical importance: Medical AI trained in one hospital often fails in another due to equipment differences — domain adaptation is critical.

Q30. Explain prototypical networks for few-shot learning. How do they classify new examples? [5 Marks]

Answer:

Prototypical Networks (Snell et al., 2017) are a simple, effective metric-based meta-learning approach for few-shot classification. Core Idea: Learn an embedding function $f_\varphi: X \to R^d$ that maps inputs to a metric space where same-class examples cluster together. Class Prototype: For each class k, compute the prototype as the mean embedding of its K support examples: $c_k = (1/K)\Sigma_{(x_i,y_i)\in S_k} f_\varphi(x_i)$. Classification: For a query example x, compute distances to all class prototypes and classify by the nearest prototype: $P(y=k|x) = \text{softmax}(-d(f_\varphi(x), c_k))$, where d is typically Euclidean distance. Training (Episodic): Sample N-way K-shot episodes from training classes. Compute prototypes for episode classes. Minimize cross-entropy loss on query examples. Update φ via SGD. At meta-test time: compute prototypes from support set, classify queries — no further training needed. Advantages: Simple and interpretable (classification = nearest prototype). No second-order gradients (unlike MAML). Naturally handles any number of shots at test time. Generalizes to zero-shot learning (use class embeddings as prototypes). Extension to Regression: Use prototypes for density estimation. Performance: Strong baseline for 5-way 1-shot and 5-way 5-shot on miniImageNet, tieredImageNet benchmarks. Often outperforms more complex methods.

Q31. What is gradient boosting? Describe how it minimizes loss using functional gradient descent. [5 Marks]

Answer:

Gradient Boosting (Friedman, 2001) views ensemble construction as optimization in function space. Objective: Minimize $L(F) = \Sigma l(y_i, F(x_i))$ over the space of additive models $F(x) = \Sigma h_t(x)$. Functional Gradient Descent: At each step t, find the direction in function space that most reduces L. This direction is the negative functional gradient — the pseudo-residuals: $r_{it} = -[\partial l(y_i,F(x_i))/\partial F(x_i)]$ evaluated at F_{t-1}. Fit a regression tree h_t to these pseudo-residuals. Find optimal step size: $\rho_t = \text{argmin}_\rho \Sigma l(y_i, F_{t-1}(x_i) + \rho h_t(x_i))$. Update: $F_t(x) = F_{t-1}(x) + v*\rho_t*h_t(x)$, where v is the learning rate (shrinkage). Key Insight: Pseudo-residuals are the direction that would most reduce the loss if we could add an arbitrary function. By fitting a tree to residuals, we greedily approximate this optimal direction. For squared error loss: $r_i = y_i - F_{t-1}(x_i)$ — literally the residuals. For other losses (log-loss, Huber): pseudo-residuals generalize the concept. Regularization: Learning rate v (smaller = more trees needed but better generalization), tree depth, subsampling (Stochastic GB). Xgboost adds L1/L2 on leaf weights, second-order approximation for faster optimization.

Q32. Explain multi-head attention. Why is it more powerful than single-head attention? [5 Marks]

Answer:

Multi-Head Attention runs h parallel attention operations (heads), each with different learned projections, then concatenates the results. Mathematical formulation: For head i: $head_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, where $W_i^Q \in R^{d_{model}\times d_k}$, $W_i^K \in R^{d_{model}\times d_k}$, $W_i^V \in R^{d_{model}\times d_v}$. Multi-head output: $\text{MultiHead}(Q,K,V) =$

Concat(head_1,...,head_h)W^O. W^O ∈ R^{hd_v×d_model} projects concatenated heads back. Typically: h=8, d_k = d_v = d_model/h, so total computation ≈ single-head with d_model. Why more powerful than single-head: (1) Different representation subspaces: Each head can attend to different types of relationships simultaneously — one head may capture syntactic dependencies, another semantic similarity, another coreference. (2) Ensemble-like effect: Multiple heads aggregate information differently, making the representation richer. (3) Expressiveness: Single head softmax collapses all information into one weighted average. Multiple heads can maintain distinct information streams. Empirical observation: In BERT, different heads specialize — some attend to previous/next words, some to coreferent words, some to syntactic governors. Analysis: Attention heads can be pruned individually — some are more important than others, suggesting functional specialization.

Q33. Compare Random Forests and XGBoost. When would you choose one over the other? [5 Marks]

Answer:

Random Forest: Parallel training (trees are independent — faster with multiple CPUs). Less prone to overfitting — averaging provides strong regularization. Fewer hyperparameters to tune. Robust to outliers and noise. Works well out-of-the-box. Lower memory usage during training. Best for: Large datasets where training speed matters, noisy data with many outliers, when you need a quick strong baseline, when interpretability via feature importance is important. XGBoost: Sequential training (trees built one at a time). Generally achieves higher accuracy on structured/tabular data — especially competitions. More expressive (can learn complex patterns over many iterations). Built-in L1/L2 regularization. Handles missing values natively. GPU acceleration available. More hyperparameters — requires careful tuning (n_estimators, learning_rate, max_depth, subsample, colsample). Can overfit if not regularized — needs early stopping. Best for: Maximum predictive accuracy on structured data, when training time is not critical, when moderate-to-low noise, Kaggle and competition settings, medium-sized datasets. Practical Guidance: Start with Random Forest as baseline (faster). If more accuracy needed → XGBoost with early stopping. XGBoost dominates tabular data benchmarks. RF preferred when data is noisy or training speed is paramount. For very large datasets, LightGBM (faster than XGBoost with leaf-wise tree growth) is often preferred.

Q34. What is dropout regularization? Explain how it works during training vs. inference. [5 Marks]

Answer:

Dropout (Srivastava et al., 2014) randomly sets neuron outputs to zero during training with probability p (the dropout rate), preventing neurons from co-adapting. Training: For each forward pass, independently for each neuron, sample a mask ri ~ Bernoulli(1-p). Apply: ỹi = ri * yi (where yi is neuron output). Effective network structure changes each mini-batch. Neurons must learn robust features useful in many contexts. Backpropagation: Gradients flow only through active (non-dropped) neurons. Implementation: Standard Dropout sets to 0 and scales outputs by 1/(1-p) during training (inverted dropout) — keeps expected value constant, so no scaling needed at inference. Inference (Testing): Dropout is disabled — all neurons are active. Equivalent to averaging predictions from 2^n implicit models (where n = number of dropout neurons). This ensemble interpretation explains why dropout reduces overfitting. This averaging is approximated cheaply using the scaling trick. Practical Details: Typical rates: p=0.1-0.3 for CNNs (spatial features more correlated), p=0.3-0.5 for fully connected layers. Dropout applied before/after activation. DropConnect drops weights instead of neurons. SpatialDropout drops

entire feature maps (better for CNNs). Variational Dropout uses the same mask across time steps (for RNNs). BatchNorm often reduces need for Dropout — they can conflict.

— SECTION C: 2-MARK QUESTIONS —

Q36. Define supervised learning. [2 Marks]

Answer:
Supervised learning trains a model on labeled data {(xi, yi)} to learn a mapping f: X → Y, minimizing prediction error. Used for classification (discrete y) and regression (continuous y).

Q37. What is a support vector in SVM? [2 Marks]

Answer:
Support vectors are the training examples closest to the decision hyperplane — they lie on the margin boundary and directly determine the hyperplane position. Only these points affect the SVM solution.

Q38. Define the bias-variance trade-off. [2 Marks]

Answer:
Total prediction error = Bias² + Variance + Irreducible Noise. Bias: error from wrong model assumptions (underfitting). Variance: error from sensitivity to training data fluctuations (overfitting). Reducing one typically increases the other.

Q39. What is K-fold cross-validation? [2 Marks]

Answer:
Dataset is split into K equal folds. Model is trained K times, each time using K-1 folds for training and 1 fold for validation. Performance is averaged over K runs for a more reliable estimate.

Q40. What is overfitting? [2 Marks]

Answer:
Overfitting occurs when a model learns the noise in the training data rather than the underlying pattern, achieving high training accuracy but poor test/generalization performance. Caused by excessive model complexity relative to data size.

Q41. Define reinforcement learning. [2 Marks]

Answer:
RL trains an agent to make sequential decisions in an environment by receiving rewards. The agent learns a policy π: S → A to maximize expected cumulative discounted reward E[$\Sigma\gamma^t r_t$].

Q42. What is a kernel function? [2 Marks]

Answer:
A kernel function $K(x_i, x_j) = \varphi(x_i)^T\varphi(x_j)$ computes the inner product in a high-dimensional feature space φ without explicitly computing the transformation — the kernel trick.

Q43. Define dropout in neural networks. [2 Marks]

Answer:
Dropout randomly deactivates neurons during training with probability p, preventing co-adaptation. At inference, all neurons are active (no dropout). Acts as implicit ensemble of 2^n models.

Q44. What is batch normalization? [2 Marks]

Answer:
Batch normalization normalizes layer inputs to zero mean and unit variance within a mini-batch, then applies learnable scale γ and shift β: $y = \gamma*(x-\mu)/\sigma + \beta$. Stabilizes training and acts as regularizer.

Q45. What is the difference between bagging and boosting? [2 Marks]

Answer:
Bagging trains models independently on bootstrap samples (parallel) — reduces variance. Boosting trains sequentially, each model correcting previous errors — reduces bias. Both create ensembles but address different error sources.

Q46. Define meta-learning. [2 Marks]

Answer:
Meta-learning ('learning to learn') trains models on multiple tasks to quickly adapt to new tasks with few examples. Goal: learn a learning algorithm/initialization, not just a specific task.

Q47. What is the self-attention mechanism? [2 Marks]

Answer:
Self-attention allows each position in a sequence to attend to all other positions: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{dk})*V$. Q, K, V are linear projections of the same input sequence.

Q48. What is transfer learning? [2 Marks]

Answer:
Transfer learning uses knowledge from a pretrained model (trained on large source data) to improve performance on a new target task with limited data, via feature extraction or fine-tuning.

Q49. Define precision and recall. **[2 Marks]**

Answer:
Precision = TP/(TP+FP) — of all predicted positives, how many are correct. Recall = TP/(TP+FN) — of all actual positives, how many are detected. F1 = 2*(P*R)/(P+R) balances both.

Q50. What is a convolutional layer? **[2 Marks]**

Answer:
A convolutional layer applies learnable filters (kernels) to input feature maps via cross-correlation, detecting local patterns. Output size: (W-F+2P)/S+1. Enables weight sharing and translational invariance.

Quick Reference Summary

Topic	Unit	Key Algorithm/Concept	Key Formula/Metric
Bias-Variance	1	Regularization, CV	Error = Bias²+Var+Noise
L1/L2 Reg.	1	Lasso/Ridge/ElasticNet	λΣ|w| / (λ/2)Σw²
Cross-Validation	1	K-Fold, Stratified, LOOCV	mean ± std of K scores
Bagging/RF	1,2	Random Forest	f(x) = (1/B)Σht(x)
Boosting	1,2	XGBoost, GBM	Ft = F_{t-1} + η*ht
SVM	2	Kernel SVM, SVR	max 2/||w||; K(xi,xj)
CNN	2	ResNet, EfficientNet	(W-F+2P)/S + 1
RNN/LSTM	2	LSTM, GRU	Ct = ft⊙C_{t-1}+it⊙C̃t
Transformer	2	BERT, GPT, T5	Attn=softmax(QKᵀ/√dk)V
Transfer Learning	2	Fine-tuning, LoRA	Feature extract / fine-tune
Meta-Learning	2	MAML, ProtoNets	θ'=θ-α∇L_task(θ)
Multi-task	2	Hard/Soft sharing	L=Σwt*Lt

— End of Study Material —